

Architecture Justification

Personal Subscription & Recurring Expense Auditor

Aditya Vats | PES1UG24CS032 | Section 5A

Architecture Selection: We chose Layered Architecture for the Personal Subscription & Recurring Expense Auditor system.

Architectural Choice

The system is organised into three layers. The Presentation Layer holds the Subscription Dashboard UI, which is what the user actually interacts with. The Business Layer contains the Subscription Detection Service, the Burn Rate Calculator, and the Alert & Notification Service, which between them handle detecting recurring charges, working out monthly spend, and sending renewal reminders. The Data Layer holds the Transaction & Subscription Database, which stores imported transactions, confirmed subscriptions, renewal dates, and any manual overrides the user has made.

Reason 1: The Logic Splits Cleanly Into Layers Already

The requirements from Lab 1 fall into three fairly obvious groups: things the user sees, things the system works out, and things that need storing. Detection, burn rate calculation, and alerting are all independent pieces of logic that operate on the same underlying transaction data, so putting them side by side in a Business Layer and giving them a shared interface to the database avoids duplicating data access code in each one. A microservices approach would have meant splitting one person's transaction data across separate services, which adds network calls and consistency problems for no real benefit on a single-user app.

Reason 2: The Detection Logic Will Change More Than Anything Else

FR-006 exists specifically because the automatic detection in FR-001 will not always get it right, which means the pattern-matching rules are the part of this system most likely to be rewritten over time. Keeping that logic inside its own component in the Business Layer means it can be improved or replaced without touching the dashboard or the database schema, as long as it keeps providing the same `ISubscriptionData` and `IRenewalDates` interfaces to the components that depend on it.

Security Advantage

The layering means the UI never talks to the database directly. Every read or write goes through the Business Layer's `IDataAccess` interface, so there is a single place where access rules can be enforced rather than having database queries scattered through the interface code. This matters here because the stored data is bank transaction history, and NFR-003 requires that data be encrypted at rest and not shared without consent. Enforcing that in one layer is far easier to verify than enforcing it in several.

Performance Benefit

Detection is the expensive operation in this system, since it has to scan roughly 90 days of transactions and group them by merchant and interval. Because it sits in its own component, its results can be computed once and then reused by both the Burn Rate Calculator and the Alert Service through their respective interfaces, instead of each of them re-scanning the raw transaction data separately. The dashboard then only reads the finished results, which keeps the screen responsive even though the underlying analysis is relatively heavy.